



Lab Assignment – 13.1

Department: CSE

Subject: DIS Lab
Code: 22CPP217

Objective: To understand the limitations of rigid relational schemas by transitioning to a document-oriented NoSQL environment. Students will perform advanced CRUD operations, leverage schema-less design for dynamic and nested data, compare this model against RDBMS architectures, and identify practical use cases where NoSQL is the preferred engineering choice.

Scenario: In one of the previous assignment, you built a rigid SQL database for *RigBuilder*. However, the engineering team has hit a roadblock: computer parts have wildly different attributes. A CPU has **cores** and **threads**, a GPU has **VRAM** and **CUDA_cores**, and a monitor has **refresh_rate** and **panel_type**. Adding all these as columns to a rigid SQL **Products** table results in massive **NULL** values and wasted space. Furthermore, user profiles now need to store multiple shipping addresses and dynamic preferences without complex multi-table joins.

You will migrate these specific microservices to a schema-less NoSQL database (MongoDB).

Resource: [MongoDB Documentation](#)

Pre-Lab Setup: Here is the Pre-Lab Setup script for the NoSQL (MongoDB) Capstone assignment.

```
SQL
// =====
// PRE-LAB SETUP: Copy and paste this into your mongosh terminal
// =====

// 1. Initialize and switch to the RigBuilder database
use rigbuilder_db;

// 2. Reset Environment: Drop existing collections to prevent duplication
errors
```

```

db.products.drop();
db.users.drop();

// 3. Insert baseline User Data (For Q2: Array & Field Manipulation)
// Notice how we are storing 'age' here, which will be removed later.
db.users.insertOne({
  name: "Aarav Sharma",
  email: "aarav.s@email.com",
  age: 24,
  account_status: "Active",
  registration_date: new Date("2026-01-15")
});

// 4. Insert baseline Product Data (For Q3: Aggregation Pipelines)
// These are simple documents. In Q1, you will insert much more complex, nested
documents.
db.products.insertMany([
  { name: "G502 Hero Gaming Mouse", price: 49.99, brand: "Logitech" },
  { name: "K70 RGB Pro Keyboard", price: 159.99, brand: "Corsair" },
  { name: "Essential Office Mouse", price: 15.00, brand: "Logitech" }, //
  Will be deleted in Q2
  { name: "Virtuoso RGB Wireless Headset", price: 209.99, brand: "Corsair" },
  { name: "Basic Mousepad", price: 9.99, brand: "Generic" } // Will be
  deleted in Q2
]);

print("Pre-Lab Setup Complete! The 'rigbuilder_db' is ready.");

```

Q 1: Breaking the Schema (Create & Read)

Your first task is to demonstrate how a document database handles polymorphic data without throwing structural errors.

1. **Dynamic Insertion:** Switch to the `rigbuilder_db` database. Insert three different hardware components into a single `products` collection.
 - Insert a CPU document containing `name`, `price`, `brand`, and a nested `specs` object (`cores: 16`, `socket: "AM5"`).

- Insert a GPU document containing `name`, `price`, `brand`, and a completely different `specs` object (`vram: "16GB"`, `ray_tracing: true`).
- Insert a Monitor document containing `name`, `price`, and `specs` (`refresh_rate: "240Hz"`, `curved: true`).
- 2. **Flexible Querying:** Write a query to retrieve *only* the products that possess a `vram` attribute, regardless of what other fields they have.
- 3. **Deep Filtering:** Write a query to find all products where the nested `specs.cores` value is strictly greater than 8.
 -

Q 2: Complex CRUD & Data Embedding (Update & Delete)

NoSQL excels at storing data together that is accessed together, eliminating the need for expensive SQL `JOIN` operations.

1. **Embedding Arrays:** Insert a new user into the `users` collection. Instead of creating a separate table for addresses (like in SQL), embed an array called `addresses` directly inside the user document, containing two different address objects (Home and Work).
2. **Array Manipulation (Update):** The user just moved. Write an update operation using the `$push` operator to add a third address object to their `addresses` array without overwriting the existing ones.
3. **Schema Evolution On-the-Fly (Update & Delete):** The marketing team decided to stop tracking `age` for users. Write an update operation using the `$unset` operator to instantly remove the `age` field from a specific user's document. Then, use `deleteOne()` to completely remove any product from the catalog that costs less than \$50.
 -

Q 3: Aggregation vs. SQL Group By

NoSQL databases use pipelines to process data, which look very different from SQL's declarative syntax.

1. **The Aggregation Pipeline:** Write an aggregation pipeline for the `products` collection that mirrors a SQL `GROUP BY`. Your pipeline must:
 - `$match`: Filter out any products priced below \$100.
 - `$group`: Group the remaining products by `brand` and calculate the average price for each brand.

Q 4: Architectural Analysis (Conceptual)

Based on your practical implementation, provide a brief written analysis comparing your NoSQL implementation to the Q1 RigBuilder SQL structure.

1. **Identify the Join-Penalty:** Explain how embedding the `addresses` array in Q2 improves read performance compared to the 3NF SQL structure you built in previous labs.
2. **Define the Limit:** Identify one specific transactional scenario in the RigBuilder platform (e.g., checking out and deducting inventory) where you would *still* prefer the RDBMS over this NoSQL model, and explain why referencing ACID properties.

NoSQL (MongoDB) Syntax Reference Guide:

SQL

1. Connecting & Database Selection

```
// Use or create a new database use
rigbuilder_db;
```

2. Creating Data (Insert)

```
// Inserting a document with nested flexible schema
db.products.insertOne({
  name: "Ryzen 9 7950X",
  price: 599.99,
  brand: "AMD",
  specs: {
    cores: 16,
    threads: 32,
    socket: "AM5"
  }
});
```

3. Reading Data (Find & Filter)

```
// Find documents where a specific nested field exists
db.products.find({ "specs.vram": { $exists: true } });

// Find with comparative operators ($gt = greater than)
db.products.find({ "specs.cores": { $gt: 8 } });
```

4. Updating Data (\$set, \$push, \$unset)

```
// Add a new address to the embedded array
db.users.updateOne(
  { name: "Aarav Sharma" },
  { $push: { addresses: { type: "Office", city: "Jaipur", zip: "302017" } } }
);

// Remove a specific field entirely from a document
db.users.updateOne(
  { name: "Aarav Sharma" },
  { $unset: { age: "" } }
);
```

5. Aggregation Pipeline (Equivalent to SQL GROUP BY)

```
db.products.aggregate([
  { $match: { price: { $gte: 100 } } },
  { $group: {
    _id: "$brand",
    average_price: { $avg: "$price" },
    total_items: { $sum: 1 }
  }}
]);
```